Torneo de Komikan Concurrente

Informe de Diseño Arquitectónico, Sincronización y Evaluación de Rendimiento

Asignatura: Sistemas Operativos (Tarea 2)
Integrante: Renato Villavicencio, Tomás Hernández
Lenguaje/Estándar: C++17 con POSIX APIs
Ejecutable: `./torneo N M`

1. RESUMEN EJECUTIVO

El presente informe documenta el diseño, la implementación y la evaluación experimental del sistema concurrente para la simulación distribuida del juego Mapuche **Komikan** en el marco de un torneo masivo con parejas concurrentes. La arquitectura fue construida sobre C++17 utilizando primitivas nativas de la interfaz POSIX (`pthread`, memoria compartida Inter-Proceso y semáforos contadores), orientada a garantizar exclusión mutua estricta, alta escalabilidad multi-mesa, aislamiento total de fallos de procesos cliente y ausencia de inconsistencias en el estado del tablero.

Mediante el patrón concurrente de **Escritor Único en Colas Acotadas**, el sistema desarticula las carreras de datos al centralizar las validaciones y mutaciones de estado en hilos especializados (*Jueces de Mesa*), mientras las piezas individuales (fichas) leen instantáneas y compiten en tiempo real. Las pruebas de estrés arrojaron un procesamiento de hasta **3 000 mov/s por mesa** con 0 inconsistencias y 100% de finalización exitosa en todas las partidas simuladas.

2. DISEÑO DEL ESQUEMA DE SINCRONIZACIÓN

2.1 Jerarquía de Procesos e Hilos

El proceso principal del servidor ejecuta la orden de arranque `./torneo N M` donde `N` representa el número total de clientes solicitantes y `M` el límite de mesas simultáneas activas. El modelo arquitectónico divide las responsabilidades entre procesos e hilos:

- **Servidor Principal (Proceso Raíz):** Asigna los recursos IPC iniciales, lanza los `N` procesos cliente mediante `fork()` y realiza el saneamiento final de recursos compartidos.
- **Hilo de Señales (Servidor):** Administra señales del sistema (como `SIGINT`) de forma síncrona mediante `sigwait()`, previniendo bloqueos o ejecuciones inseguras en manejadores asíncronos.
- **Hilo Matchmaker (Servidor):** Atiende la cola de llegadas del Lobby en orden estricto FIFO, emparejando al primer jugador (Puma) con el segundo (Perros) para asignarles una mesa disponible.
- **Hilo Juez (Servidor, ≤ M hilos activos):** Asignado dinámicamente a cada mesa activa. Es el único agente con permisos de escritura directa sobre la estructura del tablero.
- **Procesos Cliente (N procesos):** Cada jugador se ejecuta en su propio espacio de memoria aislado. Al recibir la asignación de mesa, crea hilos independientes para sus fichas: **1 hilo para el Puma** y **12 hilos para los Perros**. Los retardos de decisión de cada ficha varían entre 50 ms y 300 ms.

2.2 Memoria Compartida y Primitivas POSIX

Toda la estructura de estado —incluyendo la cola FIFO del lobby, el estado de las `M` mesas con sus tableros y los registros de clientes— reside en un único bloque de memoria compartida POSIX instanciado mediante `shm_open()` y `mmap()`. Para sincronizar múltiples procesos independientes sobre esta región, se configuraron mutexes con el atributo `PTHREAD_PROCESS_SHARED` y semáforos inicializados con `pshared = 1`.

Primitiva POSIX	Tipo y Configuración	Rol Mecánico en la Sincronización
`lobby_mtx`, `lobby_llegadas`	Mutex robusto + Semáforo contador	Protege la cola FIFO de llegada al Lobby. El Matchmaker duerme en `sem_wait` pasivamente hasta que arriben clientes.
`asignacion[i]`	Semáforo contador (1 por cliente)	Bloquea al proceso cliente `i` sin consumo de CPU hasta que el Matchmaker le asigne mesa o rechace su ingreso.
`mesas_libres`	Semáforo contador (valor inicial = M)	Garantiza el invariante de máximo `M` mesas simultáneas. Si no hay mesas, la pareja aguarda bloqueada.
`tablero_mtx` (por mesa)	Mutex robusto compartido	Protege la matriz de casillas `celda[]`, la posición `pos` y piezas capturadas. Lectura para fichas, escritura para el Juez.
`cola_mtx`, `vacios`, `llenos`	Mutex + Semáforos (Productor-Consumidor)	Canaliza las solicitudes de movimiento enviadas por las fichas (productoras) hacia el Juez de la mesa (consumidor).
`resp_sem[13]`	Arreglo de semáforos (1 por ficha)	Notificación individual de resolución del Juez hacia cada ficha: movimiento aceptado, rechazado o fin de partida.

2.3 Protocolo de Movimiento Simultáneo: Escritor Único

Para garantizar la consistencia en el tablero sin degradar el paralelismo, se adoptó el patrón de **Escritor Único**. Ninguna ficha modifica directamente las celdas del tablero. El ciclo concurrente opera de la siguiente manera:

1. **Toma de Instantánea (Ficha):** La ficha adquiere brevemente `tablero_mtx`, copia el estado del tablero (aproximadamente 170 bytes) y libera inmediatamente el lock.
2. **Evaluación Local:** Tras dormir su tiempo aleatorio (50–300 ms), la ficha calcula su intención de movimiento fuera de cualquier sección crítica.
3. **Encolado de Solicitud:** Mediante el esquema productor-consumidor (`vacios`, `cola_mtx`, `llenos`), deposita la tupla `(pieza, origen, destino)` en la cola de la mesa.

4. **Procesamiento Atómico (Juez):** El Juez extrae la solicitud en orden FIFO, adquiere `tablero_mtx`, reconvalida la validez del movimiento contra el estado real actualizado (adyacencia, casilla libre o salto en línea recta sobre un perro hacia celda vacía) y aplica los cambios atómicamente. Si el movimiento implica captura, modifica 3 casillas en una sola operación atómica.
5. **Resolución de Conflictos:** Si dos perros solicitan la misma casilla, la primera solicitud procesada la adjudica. La segunda se rechaza con la causa `"destino ocupado"`. Si la jugada se basa en una lectura antigua, se rechaza como `"estado obsoleto"`. La ficha perdedora reintenta leyendo la nueva instantánea.
6. **Verificación de Invariantes y Victoria:** Tras cada mutación, el Juez verifica la integridad estructural de la matriz, registra el paso en el log y evalúa condiciones de victoria (6 perros capturados o Puma completamente acorralado sin movimiento legal).

3. ANÁLISIS DE RIESGOS DE CONCURRENCIA

3.1 Prevención de Deadlocks (Interbloqueos)

El diseño previene interbloqueos mediante cinco salvaguardas arquitectónicas estrictas:

- **Ausencia de Anidamiento:** Ningún hilo o proceso retiene dos mutexes simultáneamente en ningún punto de la ejecución. La adquisición de locks es siempre unitaria y de corta duración, imposibilitando la formación de ciclos de espera.
- **Desacoplamiento Juez-Ficha:** La ficha libera `tablero_mtx` antes de encolar su petición. Por su parte, el Juez nunca efectúa una espera directa hacia una ficha (solo invoca `sem_post` en `resp_sem`), eliminando la retención con espera entre productor y consumidor.
- **Dimensionamiento de Cola Búfer:** La cola circular posee una capacidad física de 16 slots. Dado que la mesa contiene como máximo 13 fichas vivas (1 Puma + 12 Perros), el número de solicitudes pendientes nunca supera las 13. Por ende, las fichas productoras jamás se bloquean por falta de espacio en `vacios`.
- **Recuperación ante Muerte de Procesos (Robust Mutexes):** Si un proceso cliente es abortado (vía `kill -9`) mientras retiene un mutex compartido, la primitiva POSIX retorna el error `EOWNERDEAD` al siguiente solicitante. El hilo del servidor invoca `pthread_mutex_consistent()` y restaura el estado. Asimismo, el Juez monitorea la salud de los procesos mediante `waitpid(..., WNOHANG)` cada 100 ms (vía `sem_timedwait`). Al detectar un abandono, concede la victoria al oponente y destruye limpiamente las 17 primitivas asociadas a la mesa.
- **Apagado Limpio y Señales:** Al recibir `SIGINT`, el hilo de señales captura la instrucción síncronamente vía `sigwait()` y despierta a todo hilo bloqueado emitiendo `sem_post()` explícitos en `lobby_llegadas`, `mesas_libres`, `vacios` y `resp_sem`. Los `fork()` se ejecutan con anterioridad a la creación de hilos, evitando que los procesos hijos hereden locks tomados.

3.2 Mitigación de Livelock (Bloqueo Vivo)

La competencia entre fichas por una misma casilla se resuelve mediante el ordenamiento estricto FIFO de la cola. Al procesarse secuencialmente, se garantiza que siempre exista una solicitud ganadora, asegurando el progreso continuo del tablero. La ficha cuyo movimiento fue rechazado reevalúa su estado tras un retardo pseudoaleatorio desincronizado, lo que rompe cualquier simetría temporal. En las mediciones sobre 200 partidas simuladas, el 100% finalizó correctamente con un tiempo máximo de ejecución de 6,4 segundos.

3.3 Inanición (Starvation)

En la cola de solicitudes, cada petición aguarda como máximo la resolución de las otras 12 fichas activas, acotando el tiempo de espera. Aunque los 12 Perros generan colectivamente una tasa de solicitudes aproximadamente 12 veces superior a la del Puma, el Puma no sufre inanición pues sus peticiones son intercaladas equitativamente según su orden de llegada. En el lobby, el Matchmaker procesa las parejas respetando el orden de arribo sin postergar a ningún cliente.

4. EVALUACIÓN DE ALTERNATIVAS Y RENDIMIENTO

4.1 Matriz Comparativa de Diseño

Estrategia Evaluada	Ventajas Tácticas	Desventajas y Veredicto
A. Mutex por Casilla (Trylock en destino)	Paralelismo granular extremo a nivel de celda.	Capturas requieren bloquear 3 casillas (riesgo de deadlock por orden de locks). Compleja recuperación de 28 mutexes robustos por mesa. *Rechazada.*
B. Mutex Global (Validación en ficha)	Implementación sumamente simple.	Descentraliza la lógica de validación del juego entre fichas. Incumple la restricción formal "El Juez valida". *Rechazada.*
C. Juez Escritor Único (Cola acotada)	Centraliza reglas en `reglas_validar`. Reduce recuperación a 2 mutexes por mesa. Cero carreras de datos.	Ligera serialización de escrituras (insignificante frente al tiempo de decisión de fichas). **Estrategia Seleccionada.**

4.2 Resultados Experimentales de Rendimiento

Las pruebas se ejecutaron evaluando 100 partidas por escenario sobre un entorno distribuido simulando 20 mesas en paralelo:

ESCENARIO NORMAL (50–300 MS)

3,15 s / partida

168 movimientos promedio (53 mov/s por mesa). Solo 3 rechazos registrados de 16 851 solicitudes totales.

ESCENARIO DE ESTRÉS (0–1 MS)

3 000 mov/s / mesa

4 287 conflictos de casilla resueltos exitosamente en FIFO. 0 inconsistencias de memoria detectadas.

Dado que la validación e inserción de un movimiento por parte del Juez es una operación de complejidad `O(1)` que toma fracciones de microsegundo, la capacidad medida del Juez supera en **57 veces** la demanda requerida en condiciones normales. La latencia total del sistema está dominada por los tiempos de decisión aleatorios exigidos por la regla del juego y no por sobrecostos de sincronización POSIX.

4.3 Escalabilidad y Resiliencia de Procesos

Dado que las mesas no comparten ningún lock entre sí, el escalamiento horizontal respecto a `M` es lineal. En pruebas con `N = 200` y `M = 50`, el sistema administró dinámicamente hasta **650 hilos-ficha simultáneos** manteniendo el tope estricto de 50/50 mesas activas y 0 inconsistencias. La arquitectura de clientes basada en procesos asegura aislamiento total: un fallo crítico (`abort()` o terminación forzada) es detectado limpiamente por el servidor mediante `waitpid`, permitiendo la recolección total de recursos IPC sin afectar el resto del torneo.

> **Verificación y Pruebas de Calidad:** La suite automatizada compuesta por 40 pruebas de integración (`make test`) y la verificación extensiva con **Valgrind** (tanto en ejecución regular como durante interrupciones mediante Ctrl+C) confirmaron 0 errores de contexto, 0 bytes en uso al finalizar y la liberación completa del directorio `/dev/shm`.

4.4 Supuestos de Dominio

- **Geometría del Tablero:** Basada estrictamente en la figura reglamentaria de Komikan (red ortogonal estándar sin diagonales).
- **Mecánica de Captura:** Movimiento en línea recta sobre un perro adyacente hacia una casilla posterior vacía. Se permite una captura por turno.
- **Asignación de Roles:** En cada pareja emparejada por el Lobby, el primer cliente en arribar asume el rol del Puma.

5. CONCLUSIONES

La solución implementada demuestra la eficacia del patrón de Escritor Único apoyado en primitivas robustas de la API POSIX para resolver problemas complejos de concurrencia en tiempo real. Se logró un equilibrio óptimo entre concurrencia masiva, seguridad de memoria y resiliencia ante fallos, cumpliendo a cabalidad con todos los requerimientos de diseño y especificaciones técnicas de la asignatura.